

No Pairs Allowed

Company: JP Morgan

Round: Offline

Year: 2022

Difficulty: Easy–Medium

Topic: Strings, Greedy

Source: <https://www.designqa.in/7741/jp-morgan-interview-experiences-sde-internship-%26-fte-set-5>

Problem Description

Given an array of strings, find the minimum number of character substitutions required so that the final string contains no two adjacent characters that are the same.

A substitution means changing one character into another character.

Example

For:

["add", "book", "break"]

- add → change one d → 1
- book → change one o → 1
- break → no adjacent equal characters → 0

Therefore, the output is:

[1, 1, 0]

Input Format

- The first line contains an integer N, representing the **number of strings**.
- The next N lines contain one string each.
- Each string consists of characters that need to be checked for adjacent duplicates.

Output Format

- Print N integers, where each integer represents the **minimum number of substitutions** required for the corresponding string.
- The results should be given in the same order as the input strings.

Sample Input

```
3
add
book
break
```

Sample Output

```
1 1 0
```

Explanation

We need to check each string and make sure **no two adjacent characters are equal**.

For the input:

3
add
book
break

1. add

a d d

The two ds are adjacent and equal.

Change one d to another character:

a d x

So, **1 substitution** is required.

2. book

b o o k

The two os are adjacent and equal.

Change one o:

b o x k

So, **1 substitution** is required.

3. break

b r e a k

There are no adjacent equal characters, so no change is needed.

So, **0 substitutions** are required.

Final Output

1 1 0

1. add

a d d

The two ds are adjacent and equal.

Change one d to another character:

a d x

So, **1 substitution** is required.

2. book

b o o k

The two os are adjacent and equal.

Change one o:

b o x k

So, **1 substitution** is required.

3. break

b r e a k

There are no adjacent equal characters, so no change is needed.

So, **0 substitutions** are required.

Final Output

1 1 0

Approach to Solve This Problem:

1. Traverse each string from left to right.
2. Compare the current character with the previous character.
3. If both characters are the same, a substitution is required, so increase the count by 1.
4. After a substitution, treat the current character as different from the previous one and continue checking.
5. Repeat this until the end of the string.
6. Store the count for each string and return the results.

Java Solution:

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         for (int i = 0; i < n; i++) {
7             String s = sc.next();
8             int count = 0;
9             for (int j = 1; j < s.length(); j++) {
10                if (s.charAt(j) == s.charAt(j - 1)) {
11                    count++;
12                }
13            }
14            System.out.print(count + " ");
15        }
16        sc.close();
17    }
18 }
```

Solution:

The solution is to scan each string from left to right and check adjacent characters.

1. Start with count = 0.
2. Compare each character with the character immediately before it.
3. If both characters are the same, one substitution is required, so increase count.
4. Continue until the end of the string.
5. The final count is the minimum number of substitutions needed for that string.
6. Repeat the same process for all strings.

Complexity Analysis:

Time Complexity: $O(N \times L)$

We check each character of every string once, where N is the number of strings and L is the average length of a string.

Space Complexity: $O(1)$

We use only a few variables such as count and the loop variables. No extra data structure is required.